

ENHANCEMENT OF THE PROCESS MINING VISUALIZATION TOOL:

PETRI-NET INTEGRATION, CONVERTERS, TOKEN-GAME SIMULATION AND
HAPPY-PATH

Bachelorarbeit

Student: Jakob Schütz

Supervisor: Dipl.-Ing. Dr.techn. Marian Lux



universität
wien

AGENDA

- Motivation & Problemstellung
- Zielsetzung
- Grundlagen
- Architektur
- Demo
- Evaluation & Testing
- Herausforderung
- Zukünftige Arbeit
- Fazit



MOTIVATION & PROBLEMSTELLUNG

Große Bedeutung von Process Mining

- Transparenz & Optimierung von Prozessen

Bestehende Lösungen

- Kommerzielle Tools
 - Gute Usability
 - Eingeschränkte Erweiterbarkeit
- Open Source Tools
 - Mäßige Usability
 - Gute Erweiterbarkeit



MOTIVATION & PROBLEMSTELLUNG

Verbesserungspotential beim Petri-Netz

- Token Game ist ein Workaround
- Fehlende Silent Transitions

Vermischung von Mining- und Visualisierungslogik

- Fehlende Separation of Concerns: Mining und Visualisierung in einer Klasse



ZIELSETZUNG

01

Petri-Netz und Token Game

Implementieren eines wiederverwendbaren Petri-Netz samt Token Game und Silent Transitions.

02

Überarbeitung der Visualisierungsarchitektur

Saubere Separation der Logik zur einfachen Erweiterung.

03

Happy Path Variants

Hervorheben der häufigsten Prozesspfade, um komplexe Modelle schneller erfassbar zu machen.

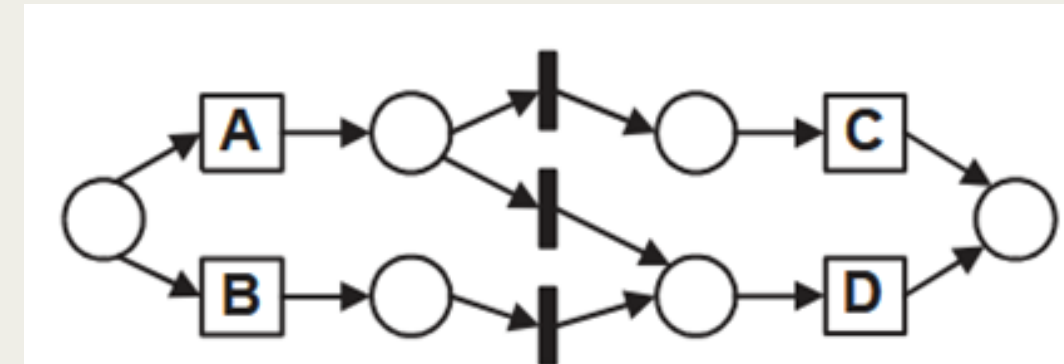


universität
wien

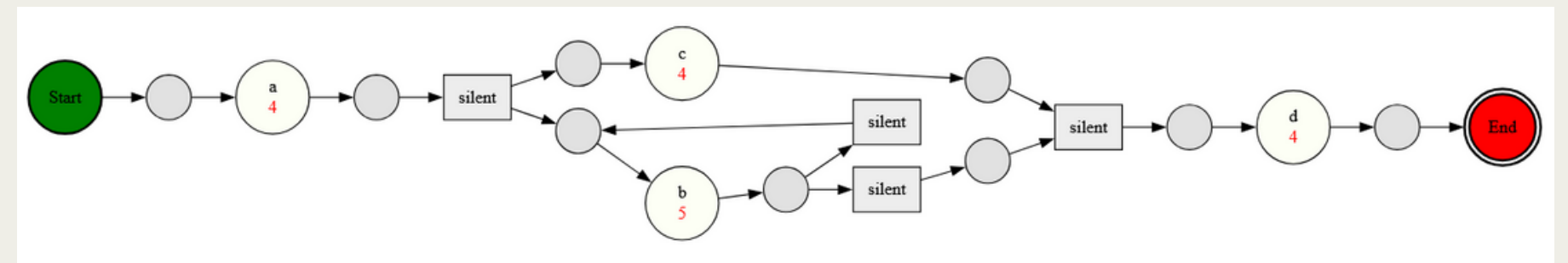
GRUNDLAGEN - PETRI-NETZ [1]

Aufbau

- **Stellen (Places)**
 - repräsentieren Zustände
- **Transitionen (Transitions)**
 - repräsentieren Aktivitäten
- **Kanten (Arcs)**
 - ermöglichen Verbindungen
- **Silent Transitions**
 - Übergänge ohne sichtbares Ereignis im Event Log



[2]



GRUNDLAGEN - TOKEN GAME [1][3]

1. Initialisierung des Petri-Netzes (Startmarkierung)
2. Auswahl eines Pfades aus dem Event Log
3. Sequentielle Verarbeitung der Events
4. Prüfung der Transitionen
5. Feuern der Transitionen
6. Abschluss des Pfades
7. Berechnung des Fitness Value



GRUNDLAGEN - PETRI-NETZ IMPLEMENTIERUNG

1. Petri-Netz Grundstruktur & Builder
 - i. Stellen, Transitionen
 - ii. Start- und Endmarkierung
2. Verbinden mittels Input- & Output Places
3. Einfügen von Silent Transitions
4. Token Game mit Fire-Logik & Token-Verteilung
5. Auslagerung in eigene Klasse
6. Integration in Alpha, Inductive & Genetic Miner

```
@staticmethod
def _fire(transitions: dict, marking: dict, transition_id: str) -> None:
    """
    Fire transition:
    remove/add token
    """
    transition = transitions[transition_id]

    # consume from input places
    for place in transition['inputs']:
        marking[place] = marking.get(place, 0) - 1

    # produce for output places
    for place in transition['outputs']:
        marking[place] = marking.get(place, 0) + 1
```

Methode zum Feuern einer Transition



GRUNDLAGEN - VISUALISIERUNGSSARCHITEKTUR

- **Trennung von Mining-und Visualisierungslogik**
 - Keine algorithmusspezifischen Visualisierungsklassen mehr
- **Konsolidierung zentraler Graphen**
 - Directly Follows
 - BPMN
 - Petri Net
- **Converter - Schicht**
 - Klare Schnittstelle zwischen Algorithmen und Visualisierung
 - Algorithmen übergeben Eingaben über dedizierte Datenklassen

```
@dataclass
class AlphaPetriNetData:
    nodes_to_draw: set[str]
    start_nodes: set[str]
    end_nodes: set[str]
    yl_set: set[tuple]
    filtered_events: set[str]
    filtered_appearance_freqs: dict[str, int]
    node_stats_map: dict[str, dict]
    edge_filter: Callable[[str, str], bool] | None = None
```

Datenklasse



universität
wien

GRUNDLAGEN - HAPPY PATH [5]

- **Grundidee**
 - Typische Abfolge von Prozessschritten
 - Referenzpfad im Prozessmodell
- **Bestimmung des Happy Path**
 - **Log-basiert:** Häufigste Varianten
 - **Normativ:** Festlegung durch Domänenwissen
 - Abweichungen möglich
- **Funktion des Happy Path**
 - Interpretationshilfe
 - Grundlage für Bewertung und Analyse
 - Gemeinsamer Grundlage für Kommunikation



GRUNDLAGEN - HAPPY PATH IMPLEMENTIERUNG

- **Grundidee**
 - Definiert als häufigste Traces im aktuell gefilterten Log
- **Implementierung**
 - Berechnung der häufigsten Traces aus dem gefilterten Event Log
 - Unterstützung mehrerer gleich häufiger Varianten
 - Absteigend sortiert nach der Länge des Traces
 - Toggle und Dropdown-Auswahl in UI
- **Limitierung**
 - Vereinfachte Darstellung



GRUNDLAGEN - HAPPY PATH

```
def get_happy_path_traces(self) -> list[tuple[str, ...]]:
    """Return all most frequent traces in the current filtered log.

    """
    source_log = self.node_frequency_filtered_log
    if not source_log:
        return []

    max_frequency = max(source_log.values())
    most_frequent_traces = [trace for trace, frequency in source_log.items() if frequency == max_frequency]
    if not most_frequent_traces:
        return []

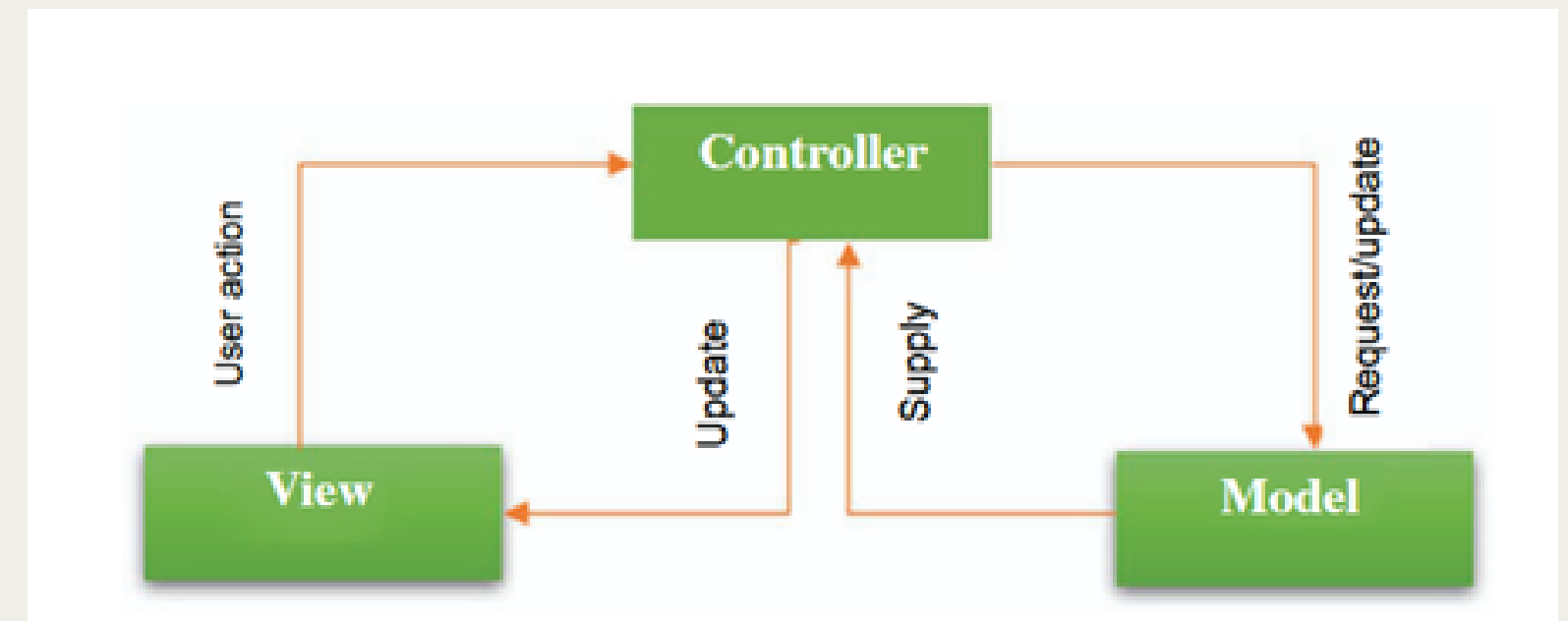
    return sorted(most_frequent_traces, key=len, reverse=True)
```



ARCHITEKTUR [4]

Umsetzung nach dem Model-View-Controller (MVC)-Prinzip

- **Model**
 - Datenaufbereitung & Mining-Algorithmen
- **View**
 - Implementiert mit Streamlit
- **Controller**
 - Verknüpfung von UI-Interaktionen und Model-Funktionen



[5]



LIVE DEMO

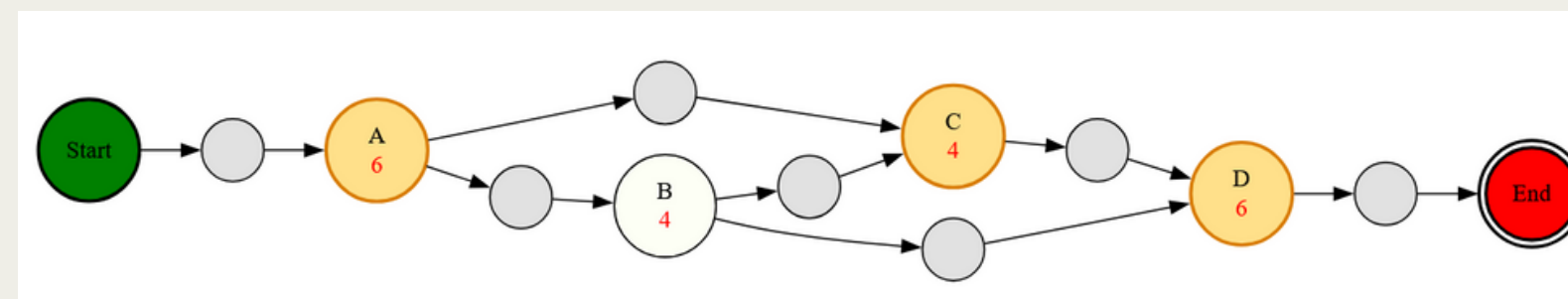


universität
wien

EVALUATION & TESTING

Einsatz von Unit-Tests und manuellen Tests

- **Petri-Netz und Token Game**
 - Anpassung bestehender Unit-Tests
 - Gegenprüfung mit Literatur
- **Visualisierungsarchitektur**
 - Existierende Unit Tests konnten wiederverwendet werden
- **Happy Path**
 - Erstellen von kleinen CSV-Logs und manuelle Überprüfung



Variante Nr. 3

	A	E
1	Case ID,Activity,Person,Timestamp	
2	1,A,John,09-3-2004:15.01	
3	1,B,Mike,09-3-2004:16.00	
4	1,D,Carol,09-3-2004:17.00	
5	2,A,John,09-3-2004:15.05	
6	2,B,Mike,09-3-2004:16.05	
7	2,D,Carol,09-3-2004:17.05	
8	3,A,Sue,09-3-2004:15.10	
9	3,C,Anne,09-3-2004:16.10	
10	3,D,Carol,09-3-2004:17.10	
11	4,A,Sue,09-3-2004:15.15	
12	4,C,Anne,09-3-2004:16.15	
13	4,D,Carol,09-3-2004:17.15	
14	5,A,John,09-3-2004:15.20	
15	5,B,Mike,09-3-2004:16.20	
16	5,C,Anne,09-3-2004:17.20	
17	5,D,Carol,09-3-2004:18.20	
18	6,A,John,09-3-2004:15.25	
19	6,B,Mike,09-3-2004:16.25	
20	6,C,Anne,09-3-2004:17.25	
21	6,D,Carol,09-3-2004:18.25	
22		

Test CSV-Log



HERAUSFORDERUNGEN

- Einarbeitung in eine bereits gewachsene und komplexe Codebasis
- Begrenzte Vorkenntnisse | Erstes großes Python Projekt
- Implementierung des Token Games als konzeptionelle Herausforderung
- Komplexe Ausführungssemantik (Places, silent Transitions, XOR- und Loop-Verhalten)



ZUKÜNFTIGE ARBEIT

- **Happy Path**
 - Visualisierung des gesamten Pfads inklusive Stellen, Kanten und Silent Transitions
- **Erweiterung der Algorithmen**
 - Implementierung neuer Mining Algorithmen
 - Führt auch zu einer größeren Zielgruppe
- **UI-Design-Verbesserungen**
 - Kontinuierliche Verbesserung des visuellen Designs weiterhin notwendig



FAZIT

- Umsetzung eines korrekten Petri-Netzes mit zugehörigem Token Game
- Implementierung einer Converter-Schicht und wiederverwendbarer Graph Klassen
- Erweiterung um Happy Path Visualisierung



LITERATUR

- [1] T. Murata, “Petri Nets: Properties, Analysis and Applications,” Proceedings of the IEEE, vol. 77, no. 4, p. 541–580, 1989.
- [2] A. K. A. de Medeiros, A. J. Weijters and W. M. P. van der Aalst, “Using genetic algorithms to mine process models: representation, operators and results,” Eindhoven University of Technology, 2004.
- [3] W. M. P. van der Aalst, . A. Weijter and A. Alves de Medeiro, “Genetic Process Mining: A Basic Approach and its Challenges,” Eindhoven University of Technology, 2005.
- [4] M. Ehsan Rana and S. S. Omar, “High assurance software architecture and design,” in System Assurances Modeling and Management, Academic Press, 2022, p. 271–285.
- [5] W. M. P. van der Aalst, “On the Pareto Principle in Process Mining, Task Mining, and Robotic Process Automation,” 2020.



Thank you!

